

Context-Aware Set Dueling for Dynamic Policy Arbitration

Diamantis Patsidis and Georgios Vavouliotis
Computing Systems Lab, Huawei Zurich Research Center

Abstract—Set Dueling (SD) [1] is an effective and widely adopted arbitration mechanism but is limited by its single-counter decision logic, relying solely on aggregate hit and miss counts of Leader Sets and ignoring richer contextual information (e.g., control-flow, sequence of memory accesses). This observation motivates our proposal, *Context-Aware Set Dueling (CASD)*, which extends the original context-oblivious SD by incorporating contextual information in the decision logic, providing a framework that can be used to design runtime arbitration mechanisms capable of selecting between competing policies. As a prototype of CASD framework, we design *DuelCeptron*, a microarchitectural prediction scheme that replaces the single-counter decision logic of SD with hashed perceptrons to make more informed and accurate arbitration decisions. To showcase the benefits of *DuelCeptron*, we apply it in a case study, showing that it significantly outperforms SD across a diverse set of 145 workloads. *DuelCeptron* is one instantiation prototype of CASD, but the broader objective of this work is to advance SD into a general-purpose, context-aware arbitration mechanism applicable across different microarchitectural domains.

I. INTRODUCTION

Modern CPUs rely on adaptive mechanisms to maintain performance across diverse and rapidly changing application domains. *Set Dueling (SD)* [1] has become a widely used primitive for such runtime arbitration and has been successfully deployed in a wide range of domains: general-purpose cache replacement policies [1], [2], prefetch-aware replacement policies [3], signature-based hit prediction [4], cache indexing [5], prefetching [6], and branch prediction [7].

The appeal of SD lies in its simplicity: a small number of Leader Sets evaluate competing policies in parallel and a single counter dictates which policy the remaining Follower Sets adopt. The Leader Sets act as controlled experiments, gathering hit and miss statistics under the competing policies without affecting the majority of the cache sets. The Follower Sets adopt the policy currently deemed superior based on the Leaders' outcomes, enabling the cache to track workload behavior and switch policies with negligible hardware cost. Its effectiveness is evident in real-world designs, as exemplified by DRRIP [2], which uses SD and has been deployed by CPU vendors to improve cache performance.

Although effective, original SD uses a context-oblivious saturating counter to dynamically select which policy the Follower Sets will adopt. This counter has a critical limitation: it reacts only after aggregate hit/miss trends accumulate, thus SD often fails to anticipate execution phase changes, leading to suboptimal policy selection. To address this issue, we propose *Context-Aware Set Dueling (CASD)*, which generalizes SD into a framework for policy arbitration based on contextual information (e.g., control flow). CASD elevates SD

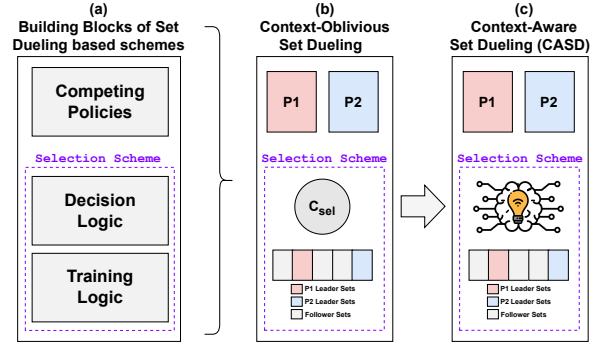


Fig. 1: (a) Building blocks of any scheme based on Set Dueling (SD), (b) original SD (Context-Oblivious Set Dueling) [1], (c) SD that exploits contextual information in the decision logic (Context-Aware Set Dueling or CASD).

to a context-aware, policy-agnostic arbitration mechanism that enables more responsive and accurate runtime adaptation.

To highlight the benefits of CASD, we design *DuelCeptron*, a prototype of CASD that uses hashed perceptron predictors to correlate contextual features with policy effectiveness and eventually select the best policy for the Follower Sets. As a case study, we apply *DuelCeptron* to a recent cache prefetching work [6] that uses original SD and show that it provides significant IPC gains over SD across 145 workloads at only 0.6KB of storage overhead. Our evaluation shows that CASD provides a scalable path for enhancing runtime policy arbitration, generalizing a technique already proven in industry.

II. CONTEXT-OBLIVIOUS SET DUELING

Figure 1 (a) illustrates that any microarchitectural scheme based on Set Dueling (SD) fundamentally consists of two components: the competing policies and the selection scheme that arbitrates among them. The selection scheme itself is composed of two building blocks: the training logic, which gathers runtime statistics from a subset of resources (e.g., Leader Sets), and the decision logic, which interprets these statistics to determine the superior policy and enforce it on the remaining resources (e.g., Follower Sets).

Figure 1 (b) shows that the original version of SD combines two competing policies, P1 and P2 by using a lightweight selection scheme. The training logic partitions the cache into dedicated Leader Sets for P1 and P2, while the remaining sets act as Follower sets, adopting the policy currently deemed superior based on the Leaders' outcomes. The decision logic is implemented with a single counter (C_{sel}), which is updated based on the relative performance on the Leader Sets. The value of C_{sel} dictates which policy the Follower Sets adopt, enabling the cache to dynamically select the superior

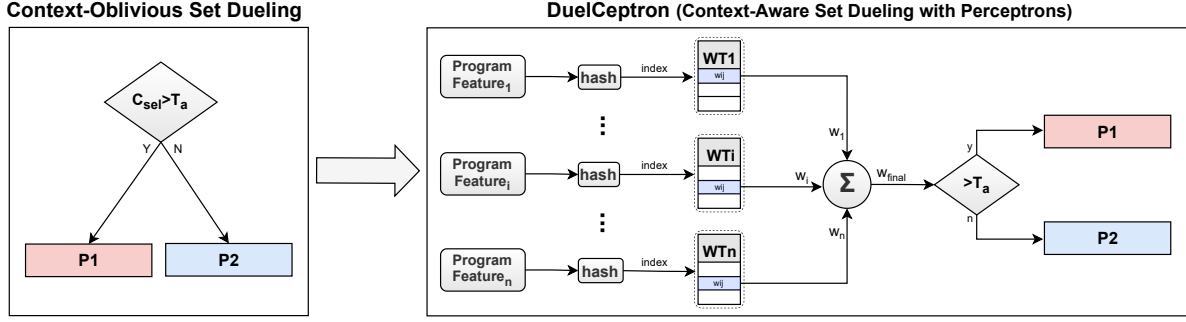


Fig. 2: Operation of original Set Dueling [1] (left) and DuelCeptron: Context-Aware Set Dueling with Perceptrons (right).

policy with minimal hardware overhead. In practice, C_{sel} is compared to an activation threshold (T_a in Figure 2 (left)). Follower Sets use P1 policy when $C_{sel} > T_a$; otherwise, P2 policy is used, as shown in Figure 2 (left). We refer to the original version of SD as Context-Oblivious Set Dueling¹ since it employs a stateless decision logic with one counter.

III. CONTEXT-AWARE SET DUELING (CASD)

This work argues that enforcing the decision logic of original SD with contextual information can provide significant IPC gains. To this end, we propose *Context-Aware Set Dueling (CASD)*. Figure 1 (c) shows that CASD retains the same training logic as the original SD, where dedicated Leader Sets track the performance of competing policies P1 and P2 and the remaining sets act as Follower Sets. The key difference between CASD and original SD lies in the decision logic: instead of a single counter, CASD uses a predictive mechanism that incorporates contextual information (e.g., control-flow, sequence of memory accesses) to determine the superior policy. By using context, CASD can anticipate shifts in behavior and make more informed policy selections, improving the responsiveness to dynamic execution phases.

In this work, we use the term context to refer to program features that capture characteristics of the current execution phase beyond the aggregate hit and miss counts of Leader Sets, as original SD does. Such features may include control-flow information (e.g., recent branch history), memory behavior (e.g., page size), temporal locality (e.g., reuse distance or access strides), and phase indicators (e.g., sequences of recently accessed PCs or addresses). By incorporating this contextual information in the decision logic, CASD scheme can move beyond simple counter-based arbitration and instead learn correlations between workload behavior and policy effectiveness. This enables CASD to adapt more precisely to diverse execution phases, ultimately improving the accuracy and responsiveness of runtime policy selection.

A. DuelCeptron: CASD with Perceptrons

This section presents DuelCeptron which is an implementation prototype of CASD (Figure 1 (c)) where the decision logic uses hashed perceptron predictors [8]. We use perceptron learning for the prototype since perceptron predictors typically have high accuracy due to their ability to correlate predictions

with multiple program features. Each perceptron predictor is realized with a Weight Table (WT) storing perceptron weights associated with one program feature. The perceptron weights are implemented with saturating counters. DuelCeptron employs a hashed perceptron predictor for each selected program feature, leveraging a diverse set of features designed using both our expertise and prior work [9], [10]. This approach ensures broad applicability across a wide range of case studies where Context-Aware Set Dueling (CASD) can be effectively employed. The feature selection process takes place offline and can use the performance, coverage, or accuracy as optimization metric, depending on the target. Figure 2 (right) presents the building blocks and the operation of DuelCeptron.

1) *Prediction*: DuelCeptron first extracts the selected program features, then hashes each feature to index its corresponding WT. The retrieved weights (w_i) are summed to produce a final weight (w_{final}) which is compared with an activation threshold T_a . If w_{final} is greater than T_a , policy P1 is used for the current access; otherwise policy P2 is chosen.

2) *Training*: To ensure correct updating of all feature weights, DuelCeptron performs training only on accesses to the Leader Sets of the competing policies, as in original SD. The difference is that, instead of incrementing or decrementing a single counter, the decision logic updates the weights associated with the selected program features, enabling more informed policy selections by correlating decisions to contextual information, thereby identifying nuanced cases that original SD misses because it relies only on global behavior, whereas DuelCeptron captures fine-grained, context-specific behavior.

IV. CASE STUDY: PAGE-SIZE AWARE PREFETCHING

As a case study, we apply DuelCeptron to a recent cache prefetching work [6] that uses original SD [1]. This work proposes a module that propagates the page size information to the lower-level cache prefetchers, enabling safe prefetching beyond 4KB physical page boundaries when the accessed block resides on a large page. We refer to a prefetcher that exploits this module as a Page Size Aware Prefetcher (Pref-PSA). This work can improve the performance of any hardware prefetcher operating at the lower-level caches.

The authors show that incorporating large pages in the design of a prefetcher may improve or harm performance. To leverage the benefits without degrading performance, they propose a composite scheme of two Pref-PSA versions that differ in one aspect: one Pref-PSA assumes 4KB pages to drive prefetching while the other Pref-PSA assumes large pages;

¹We use the terms original Set Dueling and Context-Oblivious Set Dueling interchangeably to refer to the original version of Set Dueling [1].

Physical Address	Program Counter	First Access
Last 3 Physical Addresses	Physical Page Number	Cache Line Offset
Last 3 Program Counters	Page Size	

TABLE I: List of best performing program features.

we refer to it as Pref-PSA-2MB. This composite design uses the original SD to dynamically select the better-performing prefetcher (Pref-PSA or Pref-PSA-2MB) at runtime.

A. PSA Prefetching with Original Set Dueling

This is the proposed design of the original paper [6]. The decision logic uses a single saturating counter, C_{sel} in Figure 2 (left), reflecting which prefetcher to enable for the current access. We refer to this design as *Page Size Aware Prefetcher with Set Dueling (Pref-PSA-SD)*. The operation is identical to the one presented in Section II and Figure 1 (b).

B. PSA Prefetching with DuelCeptron

This section presents the design of DuelCeptron tailored to dynamically select between Pref-PSA and Pref-PSA-2MB. The decision logic is an ensemble of N-entry WT's (Figure 2 (right)). Each WT contains weights for a single program feature (e.g., PC) and each table entry stores an M-bit saturating counter, representing the correlation between the corresponding feature and Pref-PSA and Pref-PSA-2MB. A value near the maximum (2^M) and the minimum (0) resembles as high confidence for Pref-PSA and Pref-PSA-2MB, respectively. A value near $2^M/2$ represents a weak correlation, i.e., the corresponding feature does not have high confidence for enabling either of the prefetchers. We refer to this mechanism as *Page Size Aware Prefetcher with DuelCeptron (Pref-PSA-DuelCeptron)*. The prediction and training operations are identical to those presented in Section III-A.

1) *Feature Selection*: The feature selection process takes place offline and optimizes for IPC. Based on prior research [10] and our expertise, we considered 30 features in total. First, we measure each feature's speedup in isolation and sort them by geomean IPC. Starting with the best feature, we iteratively add others if they improve geomean IPC by more than 0.3% over the current best. This continues until all features are tested, and the procedure is repeated for each prefetcher. Table I presents the 8 best-performing features. For this particular case study, using only two features (bold in Table I) is adequate to provide good performance gains. There are features that improve performance for specific workloads but their impact on geomean performance was incremental, and thus excluded from the selected set of features.

Storage Overheads. DuelCeptron uses two program features (Table I). After sensitivity analysis, we found that 512-entry WT's with 5-bit counters (per feature) provide good IPC gains. The total storage of DuelCeptron is 0.63KB (Table II).

C. Broader Applicability of CASD

Although our case study focuses on prefetching, CASD is broadly applicable since any design based on SD [1] can replace its counter-based decision logic with a context-aware predictor designed using the CASD framework. For example,

Component	Configuration
CPU Core	4GHz, 352-entry ROB, 4-wide, hashed perceptron branch predictor [11]
L1 ITLB/DTLB	64-entry, 4-way, 1-cycle, 8-entry MSHR
L2 TLB	1536-entry, 12-way, 8-cycle, 16-entry MSHR
L1 ICACHE/DCACHE	32KB/48KB, 8-way/12-way, 4-cycle/5-cycle, 8-entry/16-entry MSHR
L2 Cache	512KB, 8-way, 10-cycle, 32-entry MSHR Pref-PSA-DuelCeptron: 0.63KB (512-entry WT per feature, 5-bit counters) 32 training sets for Pref-PSA/Pref-PSA-2MB
LLC (per core)	2MB, 16-way, 20-cycle, 64-entry MSHR
DRAM	8GB, 3200MT/s

TABLE II: System configuration.

in the cache replacement domain, CASD could leverage program features to anticipate which of the competing policies of DRRIP [2] is more effective in different phases. The feature selection process (Section IV-B1) is the same across domains. We leave such exploration for future work, but we emphasize that the benefits of CASD are not limited to prefetching.

V. EVALUATION

Simulation Infrastructure. Our evaluation uses the ChampSim simulator [12], modeling an out-of-order processor with three-level cache hierarchy. Table II presents our setup. We extend ChampSim to concurrently support both 4KB and 2MB pages since the Linux THP [13] provides automatic and transparent support only for 2MB large pages.

Workloads. We evaluate a diverse set of 145 workloads from: SPEC CPU 2006 [14], SPEC CPU 2017 [15], GAP [16], LIGRA [17], PARSEC [18], and industrial workloads provided by Qualcomm for CVP1 [19]. Our evaluation focuses on workloads with an LLC MPKI > 2. All traces were obtained using the SimPoint [20] methodology, and we report the weighted mean speedups per workload. All traces except CVP1 use 250M warmup and 250M simulation instructions; CVP1 traces use 50M warmup and 100M simulation instructions.

A. Performance Evaluation

This section quantifies the IPC gains of applying DuelCeptron to the case study of Section IV. To highlight DuelCeptron's effectiveness and versatility, we apply it on a set of three state-of-the-art L2C prefetchers: Signature Path Prefetcher (SPP) [21], Variable Length Delta Prefetcher (VLDP) [22], and Perceptron-based Prefetch Filtering (PPF) [10].

We compare the IPC gains of (i) the composite scheme that uses the original SD to dynamically select between the PSA and PSA-2MB versions of the L2C prefetcher (PSA-SD), as presented in the original paper [6], and (ii) the perceptron-based composite scheme (PSA-DuelCeptron) that uses DuelCeptron to drive the activation of the PSA and PSA-2MB prefetchers, shown in Section III-A. We perform this comparison across all SPP, VLDP, and PPF prefetchers.

Figure 3 presents the IPC speedups across all considered scenarios and prefetchers. These speedups are computed over the baseline version of the corresponding prefetcher (y-axis of Figure 3), similar to the original paper [6]. Overall, PSA-DuelCeptron outperforms PSA-SD in geomean by 1.5%, 1.6%, and 1.0% when SPP, VLDP, and PPF are used as L2C prefetchers, respectively. Integrating DuelCeptron with state-of-the-art L2C prefetchers like SPP, VLDP, and PPF consistently outperforms original SD by leveraging contextual

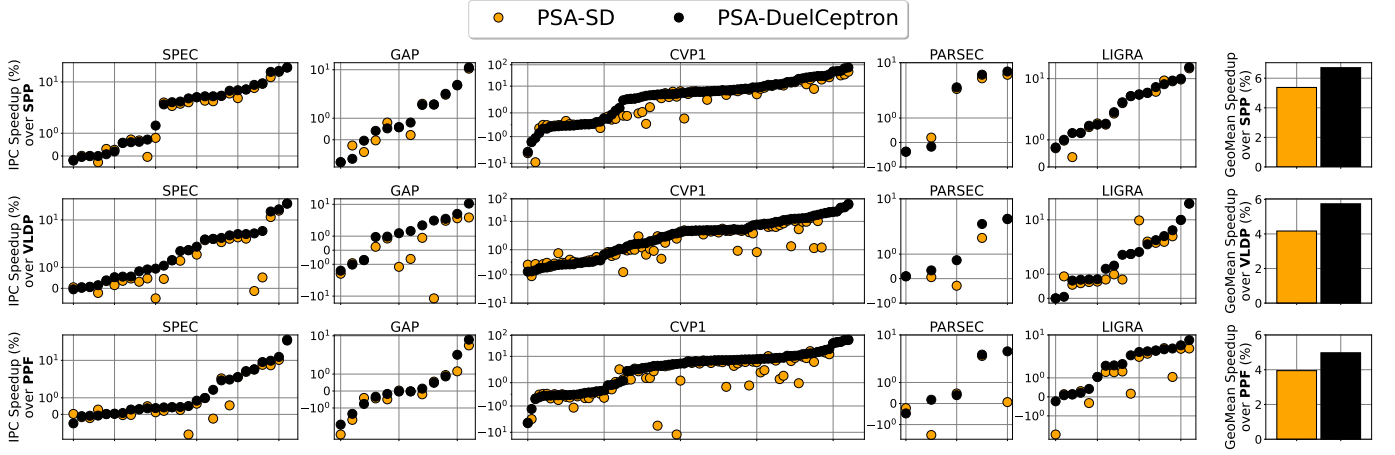


Fig. 3: Performance comparison between PSA Prefetching with Set Dueling (PSA-SD) [6] and PSA Prefetching with DuelCeptron (PSA-DuelCeptron). The IPC speedups are computed over the original version of the underlying prefetcher.

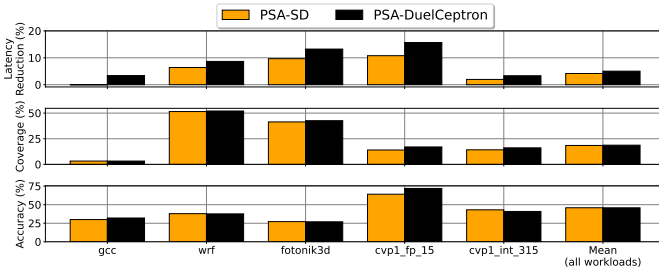


Fig. 4: Impact of PSA-SD and PSA-DuelCeptron versions of SPP on access latency, coverage, accuracy. Higher is better.

information that enables more informed and timely policy selection. Notably, for workloads with frequent phase changes, where the optimal choice alternates between Pref-PSA and Pref-PSA-2MB, DuelCeptron massively outperforms original SD. For example, DuelCeptron improves SPP’s speedup over original SD by 4.6% and 13.1% for fotonik3d and cvp1_fp_15 benchmarks, respectively. The main takeaway is that DuelCeptron outperforms original SD by leveraging contextual information for more accurate policy arbitration.

The performance gains of PSA-DuelCeptron do not originate from a single factor but rather from improvements across multiple dimensions such as timeliness, coverage, and accuracy, similar to PSA-SD [6]. Figure 4 justifies this by comparing the impact of SD and DuelCeptron on the coverage, accuracy, and timeliness of SPP prefetcher across a few representative workloads. We use L2C access latency as a metric to assess the impact on prefetching timeliness, as improved timeliness directly reduces access latency. Figure 4 shows that the IPC gains of both PSA-SD and PSA-DuelCeptron do not have a single root (e.g., better timeliness). The dominant source of benefit varies in a workload-by-workload basis. Perceptrons correlate program features with which policy (Pref-PSA vs Pref-PSA-2MB) will be beneficial for the current context, so the selection switches earlier and more precisely when needed. This is the reason why looking only at the reported averages across 145 workloads does not provide a clear understanding. For example, PSA-DuelCeptron outperforms PSA-SD for wrf benchmark mainly due to better prefetching timeliness, while for cvp1_fp_15 PSA-DuelCeptron yields better coverage, accuracy, and timeliness than PSA-SD.

VI. CONCLUSIONS

This work provides evidence that adding contextual information in designs based on Set Dueling (SD) can provide significant benefits. Context-Aware Set Dueling (CASD) generalizes the arbitration mechanism of SD by allowing any contextual information to guide policy selection. We demonstrate this through a case study, where our prototype, DuelCeptron, uses hashed perceptron predictors to outperform original context-oblivious SD with modest hardware overheads. CASD transforms SD from a counter-based heuristic into a general, context-driven primitive, opening opportunities for future adaptive arbitration mechanisms.

REFERENCES

- [1] M. K. Qureshi *et al.*, “Adaptive insertion policies for high performance caching,” *SIGARCH Computer Architecture News*, 2007.
- [2] A. Jaleel *et al.*, “High performance cache replacement using re-reference interval prediction (rip),” in *ISCA 2010*.
- [3] Wu *et al.*, “Pacman: prefetch-aware cache management for high performance caching,” in *MICRO 2011*, pp. 442–453.
- [4] C.-J. Wu *et al.*, “Ship: signature-based hit predictor for high performance caching,” in *MICRO 2011*.
- [5] K. Weston *et al.*, “A cost-effective dueling framework for set-associative cache indexing,” in *ICS 2025*, 2025.
- [6] G. Vavoulitis *et al.*, “Page size aware cache prefetching,” in *MICRO 2022*, 2022, pp. 956–974.
- [7] “Selective control flow predictor insertion,” Patent Application US 2024/0095 034 A1, 2024.
- [8] Jimenez, “Fast path-based neural branch prediction,” in *MICRO 2003*.
- [9] G. Vavoulitis *et al.*, “To cross, or not to cross pages for prefetching?” in *HPCA 2025*.
- [10] E. Bhatia *et al.*, “Perceptron-based prefetch filtering,” in *ISCA ’19*.
- [11] D. Tarjan and K. Skadron, “Merging analysis and gshare indexing in perceptron branch prediction,” *ACM Trans. Archit. Code Optim.*, 2005.
- [12] “ChampSim,” <https://csrc2.ece.tamu.edu/>.
- [13] “Transparent Huge Pages,” <http://lwn.net/Articles/423584/>.
- [14] “SPEC CPU 2006,” <https://www.spec.org/cpu2006/>.
- [15] “SPEC CPU 2017,” <https://www.spec.org/cpu2017/>.
- [16] Beamer *et al.*, “The GAP benchmark suite,” *CoRR’15*.
- [17] J. Shun *et al.*, “Ligra: A lightweight graph processing framework for shared memory,” in *PPoPP 2013*.
- [18] “PARSEC,” <https://parsec.cs.princeton.edu/>.
- [19] “Championship Value Prediction,” <https://www.microarch.org/cvp1/>.
- [20] E. Perelman *et al.*, “Using simpoin for accurate and efficient simulation,” *SIGMETRICS*, 2003.
- [21] Kim *et al.*, “Path confidence based lookahead prefetching,” in *MICRO’16*.
- [22] M. Shevgoor *et al.*, “Efficiently prefetching complex address patterns,” in *MICRO 2015*.